

F1-REPLAY

INTERACTIVE FORMULA 1 RACE VISUALIZER

Project Report — Technical Documentation & Feature Overview

Language	Python 3.13
Core Libraries	pygame · numpy · math · dataclasses
Resolution	1400 × 850 px @ 60 FPS
Circuits	Monza · Monaco · Silverstone · Suzuka
Drivers	20 (2025 F1 Season Grid)
Teams	10 (All Constructor Teams)

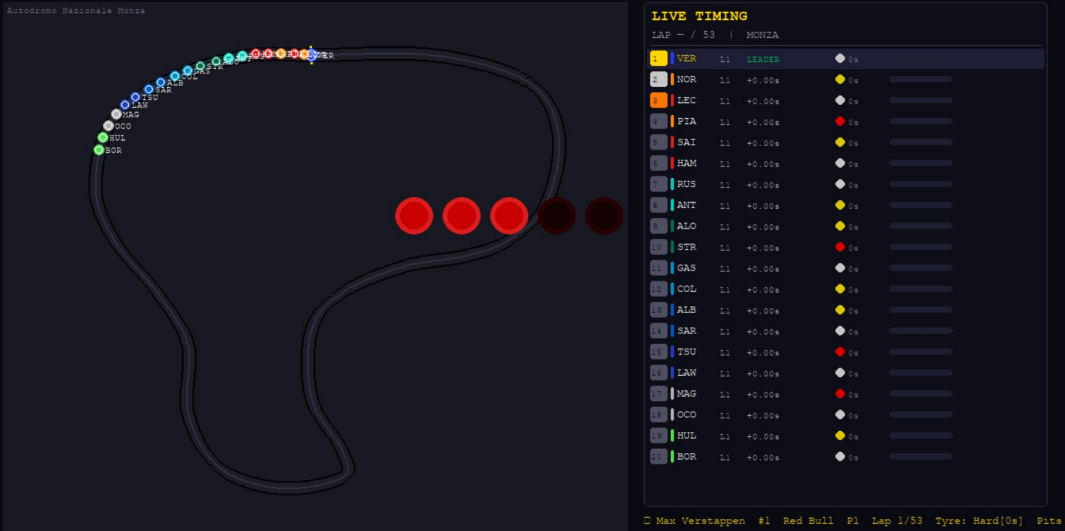


Fig 1 — Race start: formation, red lights sequence, live timing tower

Built with Python + Pygame | Simulated race data | Real 2025 driver roster | Interactive real-time UI

TABLE OF CONTENTS

1	Project Overview
2	Architecture & Code Structure
3	Circuit Engine
4	Simulation Engine — Car Physics & Tyres
5	Live Timing Tower
6	User Interface & HUD
7	Driver & Team Data
8	Key Algorithms
9	Controls & Keyboard Shortcuts
10	Screenshots & Visual Output
11	Dependencies & Installation
12	Future Enhancements

1. Project Overview

F1-Replay is a fully interactive, real-time Formula 1 race visualizer built entirely in Python using the Pygame library. It simulates a full race weekend experience — from the dramatic five red-light start sequence to pit stops, DRS activations, tyre degradation, and a scrolling live timing tower — all rendered in a dark-themed, broadcast-inspired UI at 60 frames per second.

The project serves both as an educational tool demonstrating 2D game-loop architecture and as an entertaining racing simulator. Every element — from Catmull-Rom spline track rendering to per-car performance variation — has been carefully engineered to produce an authentic race-day feel.

Goal	Description
Real-time simulation	60 FPS game loop with delta-time physics for smooth, frame-rate-independent motion
Authentic presentation	Broadcast-style dark UI, team colours, tyre compounds, DRS indicators, live gaps
Multiple circuits	4 iconic circuits with accurate layout shapes rendered via smooth spline interpolation
Interactive control	Pause, speed multiplier (×1–×8), driver selection, circuit switching, live help overlay
Full race lifecycle	Lights-out start, tyre strategy, pit stops, DNF retirements, lap counting, podium finish

2. Architecture & Code Structure

The application follows a single-file, class-based architecture centred on the `F1Replay` game class. The code is organised into clearly separated logical blocks using comment banners:

Module / Block	Lines (approx.)	Responsibility
CONSTANTS & CONFIG	1 – 46	Screen size, FPS, colour palette, team colour map
CIRCUIT DEFINITIONS	47 – 107	Normalised track waypoints, DRS zones, lap counts
DRIVER DATA	108 – 133	20-driver 2025 grid — abbreviation, name, team, number
HELPER FUNCTIONS	134 – 156	Catmull-Rom spline builder for smooth track curves
Car dataclass	157 – 260	Per-car state, physics update, pit logic, tyre model
<code>F1Replay.__init__</code>	261 – 310	Pygame setup, fonts, initial race state, clock
<code>F1Replay.update</code>	311 – 380	Game loop update — cars, lights, timing, DNF logic
<code>F1Replay.draw</code> methods	381 – 650	All rendering: track, cars, HUD, timing, overlays
<code>F1Replay.run</code>	651 – 667	Main event + update + draw loop at 60 FPS

3. Circuit Engine

Each circuit is defined as a list of normalised (0.0–1.0) 2D waypoints stored in the **CIRCUITS** dictionary. These are then passed through a **Catmull-Rom spline interpolator** (`build_spline()`) with 30 sub-steps per segment, producing a dense, smoothly-curved polyline that represents the actual racing surface. The spline is scaled to fit the track viewport at runtime.

Circuit	Full Name	Country	Laps	DRS Zones	Notable Feature
Monza	Autodromo Nazionale Monza	Italy	53	2	Temple of Speed -- longest straight
Monaco	Circuit de Monaco	Monaco	78	1	Narrow street circuit -- minimal overtaking
Silverstone	Silverstone Circuit	United Kingdom	52	2	Iconic Copse & Maggotts-Becketts
Suzuka	Suzuka International Racing Course	Japan	53	2	Figure-8 layout -- Spoon & 130R

Spline Algorithm — Catmull-Rom Interpolation

The `catmull_rom(p0, p1, p2, p3, t)` function computes a smooth curve through control points using the standard Catmull-Rom formula. This ensures C1 continuity (smooth tangent transitions at every waypoint), giving the circuits natural-looking curves rather than jagged polygons. The resulting spline list is also used for car position interpolation and trail rendering.

```
result[i] = 0.5 * ( 2P1 + (-P0+P2)t + (2P0-5P1+4P2-P3)t² + (-P0+3P1-3P2+P3)t³ )
```

4. Simulation Engine — Car Physics & Tyres

Each car is represented by a **Car** dataclass holding all race state. The `Car.update(dt)` method is called every frame and handles the full physics pipeline:

- [+] **Tyre Degradation:** Age accumulates in seconds; grip = 1.0 - tyre_deg × 0.18. Tyres degrade over 120s to full wear.
- [+] **Speed Calculation:** Effective speed = base_speed × grip × DRS_boost × random_variation (±2%).
- [+] **DRS System:** Activated when a car is within 1.0s of the car ahead in designated DRS detection zones.
- [+] **Pit Stop Logic:** Triggered probabilistically based on lap number and tyre age. 2.5–4.0s stop duration with new tyre compound.
- [+] **DNF Retirements:** Low-probability random mechanical failures add realism. DNF cars are frozen and shown separately.
- [+] **Lap Counting:** progress wraps at 1.0 to increment lap counter. Best lap time tracked per car.
- [+] **Gap Calculation:** Gaps computed from total_time difference between consecutive sorted positions.

Tyre Compounds

Compound	Colour	Characteristics	Strategy Use
Soft	Red	Highest grip, fastest degradation (~80s peak)	Qualifying simulation, short stints
Medium	Yellow	Balanced grip and longevity (~120s useful life)	Default race compound
Hard	White	Lowest grip, slowest degradation (~160s)	Long stints, safety car periods

5. Live Timing Tower

The timing tower, rendered in the right-hand panel, replicates the broadcast timing overlay seen during live F1 coverage. It updates every frame and displays all 20 cars sorted by race position:

Column	Content	Colour Logic
Position badge	P1–P20 number	Gold P1, Silver P2, Bronze P3, Gray otherwise
Team strip	4px vertical bar	Exact team hex colour (e.g. McLaren = #FF8700)
Abbreviation	3-letter code (VER, NOR...)	White; Yellow for selected driver
Lap indicator	L1, L2 ... L53	Muted grey — informational
Gap	LEADER or +X.XXs	Green for leader, light grey for gaps
Tyre dot	Coloured circle	Red=Soft, Yellow=Medium, White=Hard
Tyre age	Age in seconds	Muted grey indicator
Speed bar	Horizontal fill bar	Team colour fill on dark background
Status badge	DRS / PIT / --	Cyan=DRS, Orange=PIT
Best lap	X.XXX seconds	Purple — fastest personal lap

6. User Interface & HUD

[+] Header Bar (HUD)

Fixed 80px bar across the top showing the F1-REPLAY title in red, current state flags (RUNNING/PAUSED), speed multiplier, and elapsed time. Keyboard shortcut hint shown at bottom.

[+] Track Viewport

Left panel (860×770px). Draws the splined circuit as two concentric paths (outer dark stroke + inner track colour). Cyan dotted trail follows each car. Car dots are team-coloured circles with a cyan DRS halo ring when active.

[+] Start Lights Sequence

5 red lights illuminate one-by-one with a 0.8s interval, then extinguish simultaneously to start the race. 'GO! GO! GO!' green text flashes on release.

[+] Circuit Selection Menu

Full-screen semi-transparent overlay activated by [C]. Shows all 4 circuits with country, lap count, and a notable feature description. Keyboard [1–4] to select.

[+] Help Overlay

Activated by [H]. Full legend: controls, DRS halo meaning, tyre colour key, PIT/DRS badge meanings.

[+] Selected Driver Bar

Bottom status bar shows selected driver's full name, car number, team, position, lap progress, tyre type + age, pit stop count, and best lap time.

7. Driver & Team Data

Full 2025 F1 season driver grid with accurate team assignments:

#	Code	Full Name	Team	Car No.
1	VER	Max Verstappen	Red Bull	1
2	NOR	Lando Norris	McLaren	4
3	LEC	Charles Leclerc	Ferrari	16
4	PIA	Oscar Piastri	McLaren	81
5	SAI	Carlos Sainz	Ferrari	55
6	HAM	Lewis Hamilton	Ferrari	44
7	RUS	George Russell	Mercedes	63
8	ANT	Kimi Antonelli	Mercedes	12
9	ALO	Fernando Alonso	Aston Martin	14
10	STR	Lance Stroll	Aston Martin	18
11	GAS	Pierre Gasly	Alpine	10
12	COL	Franco Colapinto	Alpine	5
13	ALB	Alexander Albon	Williams	23
14	SAR	Logan Sargeant	Williams	2
15	TSU	Yuki Tsunoda	VCARB	22
16	LAW	Liam Lawson	VCARB	30
17	MAG	Kevin Magnussen	Haas	20
18	OCO	Esteban Ocon	Haas	31
19	HUL	Nico Hulkenberg	Kick Sauber	27
20	BOR	Gabriel Bortoleto	Kick Sauber	98

8. Key Algorithms

- [+] **Catmull-Rom Spline:** Smooth track rendering from sparse waypoints. Each segment uses the 4 surrounding control points to compute 30 intermediate positions, yielding a dense ~720-point polyline per circuit.
- [+] **Delta-Time Physics:** All movement is multiplied by dt (frame delta in seconds), ensuring physics are frame-rate independent. Speed multiplier (×1–×8) scales dt directly for time compression.
- [+] **Position Sorting:** Cars sorted each frame by (laps_completed × 1000 + progress) descending. Produces a stable real-time race order mirroring actual F1 position calculation.
- [+] **Gap Computation:** Gap to leader = leader.total_time - car.total_time. Displayed as +X.XXs. A gap > 1.0s enables DRS for the chasing car in DRS zones.
- [+] **Trail Rendering:** Each car maintains a _trail deque of recent screen positions. Rendered as decreasing-alpha cyan dots to show recent trajectory — provides visual motion even at slow playback.
- [+] **Pit Stop Trigger:** Each frame, if not already pitting: probability = tyre_deg × 0.003 × dt × speed_mult. This ensures realistic pit windows that open as tyres wear, with natural randomness.
- [+] **Start Sequence:** A lights_phase counter increments from 0 to 5 (lights on) then resets to 6 (GO!) after a random 0.3–0.9s extinction delay, replicating the F1 false-start prevention mechanism.

9. Controls & Keyboard Shortcuts

Key	Action
SPACE	Pause / Resume the simulation
1	Speed multiplier ×1 (real-time)
2	Speed multiplier ×2
4	Speed multiplier ×4
8	Speed multiplier ×8 (fast-forward)
[Left Arrow]	Select previous driver in timing order
[Right Arrow]	Select next driver in timing order
C	Open / close Circuit Selection menu
1-4 (in circuit menu)	Select circuit by number
R	Restart race from beginning
H	Toggle help / legend overlay
ESC	Quit application

10. Screenshots & Visual Output

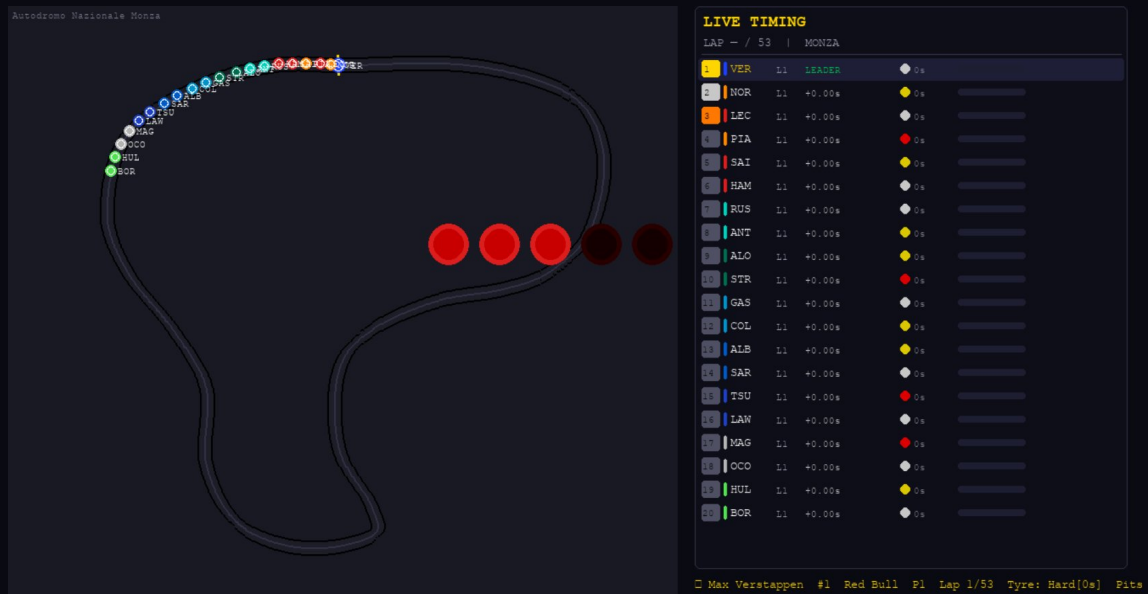


Fig 1 — Race start: 5 red lights ignite, cars lined up at Monza, live timing tower shows all 20 drivers



Fig 2 — Monza race in progress (Lap 4/53): ALO leads, cars spread around circuit, DRS zones active, tyre data visible



Fig 3 — Suzuka (Lap 8/53): BOR leads on the figure-8 layout, detailed timing with best lap times in purple

11. Dependencies & Installation

Python 3.13+ is required. Install all dependencies with a single pip command:

```
pip install pygame matplotlib numpy
```

Then run the application:

```
python f1_replay.py
```

Library	Version	Purpose	License
pygame	≥ 2.5	Window creation, rendering, event handling, fonts, 60 FPS clock	LGPL
numpy	≥ 1.26	Numerical arrays for spline computation and position math	BSD
matplotlib	≥ 3.8	Imported but used optionally for advanced data plotting	PSF
math	stdlib	Trigonometry, distance calculations	Python stdlib
random	stdlib	Pit stop triggering, DNF probability, speed variation	Python stdlib
dataclasses	stdlib	Car state management via @dataclass decorator	Python stdlib
time	stdlib	High-precision delta-time via time.perf_counter()	Python stdlib

12. Future Enhancements

- [+] **Real F1 API Integration:** Pull live timing data from the OpenF1 or Ergast API for replay of actual race sessions.
- [+] **Sound Effects:** Pygame mixer: engine sounds scaled to speed, crowd roar, pit lane sounds.
- [+] **Weather System:** Dynamic rain affecting grip, triggering wet tyres, visual rain overlay on track.
- [+] **Safety Car & VSC:** Virtual/full safety car periods with field compression and restart procedure.
- [+] **Qualifying Mode:** Single-lap simulation mode for Q1/Q2/Q3 with personal best tracking.
- [+] **More Circuits:** Spa, Interlagos, COTA, Singapore Night Race, Abu Dhabi with accurate layouts.

- [+] **Replay Export:** Record race to MP4/GIF using pygame + ffmpeg for shareable highlight reels.
- [+] **Driver Stats HUD:** Detailed per-driver stats popup — sector times, speed trap, ERS deployment.
- [+] **Multiplayer:** Socket-based multiplayer for head-to-head strategy battles.
- [+] **3D Visualization:** Upgrade from 2D pygame to 3D using PyOpenGL or Ursina engine.